COMPUTER FUNDAMENTALS | STUDENT EDITION

Introduction to Computers

Clear, module-wise study notes

Definitions, diagrams, examples, conversions, and exam practice

Simple explanation first. Technical detail next. Then a worked example.

Follow each module roadmap and practise the examples on paper.

48 SYLLABUS TOPICS | 115 PAGES | 3 MODULES

How to get the most from these notes

A simple study routine

- Read the plain-language definition first; then reread the technical definition and notation.
- Follow the process map with your finger. For number conversions, write each intermediate step yourself.
- Cover the worked answer and predict the result before checking it.
- Finish each topic by answering the viva cue aloud and reviewing the common mistakes.

What is inside

- The notes follow all three supplied modules and preserve the syllabus section order and terminology.
- Every topic is split into two pages: concept/definition and example/exam help.
- Module transition pages, concise definitions, high-yield review maps, practice questions, and source notes appear at the end.
- Historical dates and textbook groupings are explained carefully where conventions vary.

How to answer in exams and viva

- Define the term in one direct sentence.
- Name the parts, rule, or steps using correct technical words.
- Use one small example, labelled diagram, or conversion trace.
- State the result and one relevant limitation or comparison.

QUICK RECALL

Small worked examples make computer fundamentals much easier to remember.

Module 1 - what to study and in what order

Module goal

Understand what a computer does, how its parts work together, and how computers and software are classified.

Syllabus map - exact section order

- Introduction to computers: Data and Information; Computer: Definition and Main Features; Applications of Computers; Generations of Computers and Their Features.
- Basic Computer Organization: Basic Operations of a Computer; Anatomy of a Computer System; Input/Output Units and Their Functions; Storage Units and Their Functions; Types of Storage and Their Functions.
- Hardware & Software: Hardware: Definition and Types; Software: Definition and Types; Relationship Between Hardware and Software.
- Classification of Computers: Classification of Computers by Mode of Use and Features.

Recommended study order

- Begin with data, information, the computer definition, its features, applications, and generations.
- Trace the basic operations through input, CPU, memory, output, and storage.
- Finish by distinguishing hardware/software and comparing types of computers.

QUICK RECALL

Before moving on, draw the input-process-output-storage model and explain the job of each major component.

TOPIC 01 / 48 | Introduction to computers
CONCEPT AND DEFINITIONS

Data and Information

Simple definition

Data are raw facts; information is data organized and interpreted so it helps answer a question.

Formal / technical definition

Data are symbols or observations that can be recorded and processed. Information is data interpreted in a context so that it has meaning or supports a decision. The distinction depends on purpose and context.

Notation and how to read it

data + context + processing -> information; information can become data for a later process

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
Raw values -> validation/organization/calculation -> meaningful result -> decision
```

QUICK RECALL

Keep this in mind: Define both terms and transform one small set of raw values into a useful result.

TOPIC 01 / 48 | Introduction to computers
EXAMPLE, COMMON ERRORS AND EXAM HELP

Data and Information

Worked example / application

Raw data: 72, 81, 65. Context: marks out of 100. Information: average = (72+81+65)/3 = 72.67, which summarizes performance.

Important distinction / practical note

The same value can be data in one task and information in another; information quality depends on accuracy, relevance, and timeliness.

Common mistakes to avoid

- Do not call every recorded value useful information without context.
- An average can hide important variation; choose a summary that fits the question.

Quick viva cues

- Core viva answer: Define both terms and transform one small set of raw values into a useful result.
- Likely follow-up: The same value can be data in one task and information in another; information quality depends on accuracy, relevance, and timeliness.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Define both terms and transform one small set of raw values into a useful result.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 02 / 48 | Introduction to computers CONCEPT AND DEFINITIONS

Computer: Definition and Main Features

Simple definition

A computer is an electronic, programmable system that accepts data, follows instructions, produces results, and can store them.

Formal / technical definition

A computer system accepts input, processes it under a stored program, produces output, and stores data/results. Common characteristics include speed, accuracy when instructions/data are correct, repeatability, storage capacity, automation, and versatility.

Notation and how to read it

Input -> processing under instructions -> output; storage supports later use

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
Input 24,18 -> CPU executes ADD -> output 42 -> optional saved result
```

QUICK RECALL

Keep this in mind: State the input-process-output-storage model and explain two features with their limits.

TOPIC 02 / 48 | Introduction to computers EXAMPLE, COMMON ERRORS AND EXAM HELP

Computer: Definition and Main Features

Worked example / application

A calculator program reads 24 and 18, applies the addition instruction, displays 42, and can store the result in memory or a file.

Important distinction / practical note

A computer does not understand whether an answer is sensible by itself; incorrect input or instructions can produce incorrect output (GIGO).

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: State the input-process-output-storage model and explain two features with their limits.
- Likely follow-up: A computer does not understand whether an answer is sensible by itself; incorrect input or instructions can produce incorrect output (GIGO).

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: State the input-process-output-storage model and explain two features with their limits.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 03 / 48 | Introduction to computers
CONCEPT AND DEFINITIONS

Applications of Computers

Simple definition

Computers are used wherever information must be created, analyzed, communicated, controlled, or stored.

Formal / technical definition

Applications span data processing, scientific/engineering computation, communication, education, healthcare, finance, commerce, public services, design, entertainment, and industrial control. The suitable system depends on workload, reliability, latency, privacy, and cost requirements.

Notation and how to read it

application = task + input/data + processing + output + users

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
Sensor / user input -> computer system -> analysis or control -> report / action
```

QUICK RECALL

Keep this in mind: For any two fields, identify the task, data, processing, and output.

TOPIC 03 / 48 | Introduction to computers

EXAMPLE, COMMON ERRORS AND EXAM HELP

Applications of Computers

Worked example / application

Banking: update account transactions; hospital: retrieve patient records and process images; factory: monitor sensors and control machines; education: create, simulate, and share learning material.

Important distinction / practical note

Automation can improve consistency but does not remove the need for human judgment, secure data handling, and error checks.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: For any two fields, identify the task, data, processing, and output.
- Likely follow-up: Automation can improve consistency but does not remove the need for human judgment, secure data handling, and error checks.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: For any two fields, identify the task, data, processing, and output.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 04 / 48 | Introduction to computers
CONCEPT AND DEFINITIONS

Generations of Computers and Their Features

Simple definition

Computer generations are a teaching framework that groups eras by the main hardware technology used to build computers.

Formal / technical definition

The commonly taught five-generation scheme describes broad shifts: vacuum tubes; transistors; integrated circuits; microprocessors and VLSI; and ongoing parallel/AI-oriented systems. Dates and labels are approximate; technologies overlapped rather than switching everywhere at once.

Notation and how to read it

1 vacuum tube -> 2 transistor -> 3 integrated circuit -> 4 microprocessor/VLSI -> 5 parallel and AI-oriented systems

How it works

- Identify the main hardware technology used to classify the generation.
- Connect the technology to size, speed, power, cost, and reliability trends.
- Treat dates and boundaries as approximate because textbooks group them differently.

Diagram / process map / worked structure

```
Technology becomes more integrated -> systems generally become smaller, faster, cheaper per operation, and more capable
```

QUICK RECALL

Keep this in mind: Make a comparison table with hardware technology, typical size/cost, software/language trend, and example class.

TOPIC 04 / 48 | Introduction to computers EXAMPLE, COMMON ERRORS AND EXAM HELP

Generations of Computers and Their Features

Worked example / application

1st generation (1940s-1950s, approximate): vacuum tubes; very large, hot, costly, power-hungry; machine language and punched cards.

2nd (1950s-1960s): transistors; smaller, faster, more reliable; magnetic-core memory, assembly and early high-level languages.

3rd (1960s-1970s): integrated circuits; improved size, speed, and cost; operating systems, multiprogramming, keyboards/monitors.

4th (from 1970s): microprocessors and VLSI; personal computers, networks, graphical interfaces, large-scale storage.

5th (label varies by text): parallel processing and AI-oriented systems such as expert systems and natural-language work.

Important distinction / practical note

The five-generation scheme is a useful classroom summary, not a precise universal chronology; fifth-generation definitions vary by textbook.

Common mistakes to avoid

- Generation dates vary by textbook and overlap during transitions.
- Do not confuse the main hardware technology with a computer's size category.

Quick viva cues

- Core viva answer: Make a comparison table with hardware technology, typical size/cost, software/language trend, and example class.
- Likely follow-up: The five-generation scheme is a useful classroom summary, not a precise universal chronology; fifth-generation definitions vary by textbook.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Make a comparison table with hardware technology, typical size/cost, software/language trend, and example class.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 05 / 48 | Basic Computer Organization
CONCEPT AND DEFINITIONS

Basic Operations of a Computer

Simple definition

A computer accepts input, keeps data/instructions in memory, processes data, gives output, and stores results for later.

Formal / technical definition

The input-process-output-storage cycle is coordinated by control operations. At the hardware level, CPU fetches an instruction, decodes it, executes it, and stores or communicates the result; peripheral and memory operations are coordinated through buses/controllers.

Notation and how to read it

Input -> Memory -> CPU processing/control -> Output; storage provides persistence

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
Input -> [Memory <-> CPU: Control Unit + ALU + Registers] -> Output
<-> Secondary Storage
```

QUICK RECALL

Keep this in mind: Trace one calculation through input, memory, CPU, output, and optional storage.

TOPIC 05 / 48 | Basic Computer Organization

EXAMPLE, COMMON ERRORS AND EXAM HELP

Basic Operations of a Computer

Worked example / application

To add two numbers: input values -> store operands -> fetch/decode ADD -> ALU adds -> write result -> display or save.

Important distinction / practical note

The cycle is conceptual; real systems overlap operations using caches, pipelines, DMA, and multiple cores.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Trace one calculation through input, memory, CPU, output, and optional storage.
- Likely follow-up: The cycle is conceptual; real systems overlap operations using caches, pipelines, DMA, and multiple cores.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Trace one calculation through input, memory, CPU, output, and optional storage.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 06 / 48 | Basic Computer Organization
CONCEPT AND DEFINITIONS

Anatomy of a Computer System

Simple definition

A computer system combines hardware, software, data, and users/procedures; the main hardware parts communicate to perform work.

Formal / technical definition

A simplified computer architecture includes CPU (control unit, ALU, registers), primary memory, input/output devices, secondary storage, and interconnection paths such as buses. The CPU executes instructions; memory supplies active instructions/data; peripherals communicate through interfaces/controllers.

Notation and how to read it

CPU = CU + ALU + registers; system = CPU + memory + I/O + storage + interconnect

How it works

- Identify the component and the job it performs.
- Trace data/control signals between input, processing, memory, output, and storage.
- Distinguish a physical component from the software instructions that control it.

Diagram / process map / worked structure

```
+------------------+
Input devices -> | CPU <-> Memory | -> Output devices
+--------+---------+
<-> Storage
```

QUICK RECALL

Keep this in mind: Label CPU subparts, primary memory, storage, input, output, and the paths between them.

TOPIC 06 / 48 | Basic Computer Organization
EXAMPLE, COMMON ERRORS AND EXAM HELP

Anatomy of a Computer System

Worked example / application

When a user opens a file: storage supplies file blocks, memory holds active portions, CPU executes the program, display presents output, and keyboard/mouse provide input.

Important distinction / practical note

The processor is not the whole computer: storage, memory, I/O, interconnects, software, and power/cooling matter too.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Label CPU subparts, primary memory, storage, input, output, and the paths between them.
- Likely follow-up: The processor is not the whole computer: storage, memory, I/O, interconnects, software, and power/cooling matter too.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Label CPU subparts, primary memory, storage, input, output, and the paths between them.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 07 / 48 | Basic Computer Organization CONCEPT AND DEFINITIONS

Input/Output Units and Their Functions

Simple definition

Input units send data or commands into a computer; output units present the computer's results in a form people or other devices can use.

Formal / technical definition

An I/O subsystem includes peripheral devices, device interfaces/controllers, drivers, and transfer mechanisms. Input converts external signals/actions into machine-usable data; output converts processed results into visual, audio, printed, or machine-control signals. Many devices perform both directions.

Notation and how to read it

input device -> interface/controller -> system; system -> interface/controller -> output device

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
Human/environment -> device -> encoded signals -> computer
computer -> encoded result -> device -> human/environment
```

QUICK RECALL

Keep this in mind: Give at least three examples and distinguish physical device from its controller/driver.

TOPIC 07 / 48 | Basic Computer Organization EXAMPLE, COMMON ERRORS AND EXAM HELP

Input/Output Units and Their Functions

Worked example / application

Keyboard: input characters; scanner: input images; monitor: visual output; printer: hard-copy output; touchscreen and network adapter: input and output.

Important distinction / practical note

A touchscreen, storage drive, or network interface can both receive and send data; input/output is a role, not always a one-way device category.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Give at least three examples and distinguish physical device from its controller/driver.
- Likely follow-up: A touchscreen, storage drive, or network interface can both receive and send data; input/output is a role, not always a one-way device category.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Give at least three examples and distinguish physical device from its controller/driver.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 08 / 48 | Basic Computer Organization
CONCEPT AND DEFINITIONS

Storage Units and Their Functions

Simple definition

Storage keeps data and instructions. Small, fast storage supports current work; larger, slower storage keeps files for later.

Formal / technical definition

Computer storage is organized hierarchically. Registers and caches support immediate CPU work; RAM holds active programs/data; persistent devices retain data without power. Units commonly use bit, byte, and larger byte multiples; conventions may be decimal (kB=1000 bytes) or binary (KiB=1024 bytes).

Notation and how to read it

1 nibble=4 bits; 1 byte=8 bits; 1 KiB=1024 bytes; 1 MiB=1024 KiB; 1 GiB=1024 MiB

How it works

- Classify the medium by volatility, capacity, speed, cost, and access pattern.
- Trace a read or write from the CPU through the memory/storage hierarchy.
- Choose the unit carefully and distinguish bits, bytes, and binary prefixes.

Diagram / process map / worked structure

```
Registers -> Cache -> RAM -> SSD/HDD -> archival/removable storage
Typical trend: capacity increases; access speed generally decreases
```

QUICK RECALL

Keep this in mind: State the unit convention, convert step-by-step, and compare speed, capacity, and persistence.

TOPIC 08 / 48 | Basic Computer Organization
EXAMPLE, COMMON ERRORS AND EXAM HELP

Storage Units and Their Functions

Worked example / application

A 4 KiB block contains 4 x 1024 = 4096 bytes under binary-unit notation. A 1-byte ASCII character can represent one code unit for basic ASCII text, but encodings such as UTF-8 may use multiple bytes per character.

Important distinction / practical note

Some manufacturers use KB/MB/GB as decimal units. Check whether a problem specifies kB/MB or KiB/MiB.

Common mistakes to avoid

- A bit is not a byte; one byte is commonly 8 bits.
- Decimal kB and binary KiB are different units: 1000 bytes vs 1024 bytes.

Quick viva cues

- Core viva answer: State the unit convention, convert step-by-step, and compare speed, capacity, and persistence.
- Likely follow-up: Some manufacturers use KB/MB/GB as decimal units. Check whether a problem specifies kB/MB or KiB/MiB.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: State the unit convention, convert step-by-step, and compare speed, capacity, and persistence.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 09 / 48 | Basic Computer Organization
CONCEPT AND DEFINITIONS

Types of Storage and Their Functions

Simple definition

Primary storage is used directly during processing; secondary storage keeps larger amounts of data more permanently.

Formal / technical definition

Primary storage includes CPU registers, cache, RAM, and ROM/firmware storage. Secondary storage includes SSDs, hard disks, optical media, and removable flash devices. RAM is typically volatile; secondary storage is non-volatile, though availability and durability depend on the device and protection strategy.

Notation and how to read it

Primary: registers/cache/RAM/ROM; secondary: SSD/HDD/optical/flash; volatile vs non-volatile describes power retention

How it works

- Classify the medium by volatility, capacity, speed, cost, and access pattern.
- Trace a read or write from the CPU through the memory/storage hierarchy.
- Choose the unit carefully and distinguish bits, bytes, and binary prefixes.

Diagram / process map / worked structure

```
CPU registers/cache <-> RAM <-> storage device
ROM/firmware supports startup and hardware initialization
```

QUICK RECALL

Keep this in mind: Compare each storage type by CPU access, speed, capacity, volatility, and purpose.

TOPIC 09 / 48 | Basic Computer Organization EXAMPLE, COMMON ERRORS AND EXAM HELP

Types of Storage and Their Functions

Worked example / application

An open program is loaded from SSD into RAM; CPU cache stores frequently used data; firmware in ROM/flash helps start the machine; files remain on the SSD after shutdown.

Important distinction / practical note

ROM is a broad historical term; modern firmware is commonly stored in rewritable non-volatile flash memory.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Compare each storage type by CPU access, speed, capacity, volatility, and purpose.
- Likely follow-up: ROM is a broad historical term; modern firmware is commonly stored in rewritable non-volatile flash memory.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Compare each storage type by CPU access, speed, capacity, volatility, and purpose.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 10 / 48 | Hardware & Software
CONCEPT AND DEFINITIONS

Hardware: Definition and Types

Simple definition

Hardware is the physical equipment that can be touched: processor, memory chips, keyboard, display, storage, and network devices.

Formal / technical definition

Hardware comprises physical electronic, electromechanical, and optical components. It may be grouped as processing, primary memory, secondary storage, input, output, communication, and interconnection hardware.

Notation and how to read it

Hardware classes: processing | memory | storage | input | output | communication

How it works

- Identify the component and the job it performs.
- Trace data/control signals between input, processing, memory, output, and storage.
- Distinguish a physical component from the software instructions that control it.

Diagram / process map / worked structure

```
device -> controller/interface -> bus/interconnect -> CPU/memory or external network
```

QUICK RECALL

Keep this in mind: Classify a list of components by role and give each component's function.

TOPIC 10 / 48 | Hardware & Software

EXAMPLE, COMMON ERRORS AND EXAM HELP

Hardware: Definition and Types

Worked example / application

CPU executes instructions; RAM holds active data; SSD stores files; keyboard sends input; monitor presents output; NIC communicates over a network.

Important distinction / practical note

Hardware capability alone does not specify how a task should be performed; programs and configuration provide that behavior.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Classify a list of components by role and give each component's function.
- Likely follow-up: Hardware capability alone does not specify how a task should be performed; programs and configuration provide that behavior.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Classify a list of components by role and give each component's function.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 11 / 48 | Hardware & Software

CONCEPT AND DEFINITIONS

Software: Definition and Types

Simple definition

Software is the set of programs and related instructions that tell hardware what work to perform.

Formal / technical definition

Software is executable code and associated data/configuration. System software manages hardware and provides a platform; application software performs user tasks; utility/development tools maintain or create systems. Firmware is software stored in non-volatile hardware memory.

Notation and how to read it

Software categories: system | application | utility | programming/development tools; firmware bridges code and hardware

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
User task -> application -> OS/services/drivers -> hardware
```

QUICK RECALL

Keep this in mind: Define software and classify examples by the job each one performs.

TOPIC 11 / 48 | Hardware & Software

EXAMPLE, COMMON ERRORS AND EXAM HELP

Software: Definition and Types

Worked example / application

System: operating system and device driver. Application: word processor or browser. Utility: backup/compression tool. Development: compiler/editor.

Important distinction / practical note

Categories can overlap by purpose: a tool may be bundled with an OS or delivered as an application.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Define software and classify examples by the job each one performs.
- Likely follow-up: Categories can overlap by purpose: a tool may be bundled with an OS or delivered as an application.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Define software and classify examples by the job each one performs.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 12 / 48 | Hardware & Software
CONCEPT AND DEFINITIONS

Relationship Between Hardware and Software

Simple definition

Hardware provides the working parts; software provides instructions. Each depends on the other to turn a task into a result.

Formal / technical definition

A computer system executes software through processor instructions and hardware interfaces. The OS abstracts devices, drivers translate general requests into device-specific operations, and hardware performs physical signal processing/storage/communication.

Notation and how to read it

application -> OS -> driver/interface -> hardware -> result/status back to software

How it works

- Identify the component and the job it performs.
- Trace data/control signals between input, processing, memory, output, and storage.
- Distinguish a physical component from the software instructions that control it.

Diagram / process map / worked structure

```
Task request -> application logic -> OS/driver translation -> physical action -> feedback
```

QUICK RECALL

Keep this in mind: Explain the relationship using one complete example such as printing or saving a file.

TOPIC 12 / 48 | Hardware & Software EXAMPLE, COMMON ERRORS AND EXAM HELP

Relationship Between Hardware and Software

Worked example / application

A print request goes from a document application to the OS print service, through a printer driver, over a connection, to the printer hardware.

Important distinction / practical note

The same hardware can perform different tasks with different software; software cannot run without a compatible execution platform.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Explain the relationship using one complete example such as printing or saving a file.
- Likely follow-up: The same hardware can perform different tasks with different software; software cannot run without a compatible execution platform.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Explain the relationship using one complete example such as printing or saving a file.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 13 / 48 | Classification of Computers
CONCEPT AND DEFINITIONS

Classification of Computers by Mode of Use and Features

Simple definition

Computers can be grouped by the kind of data they handle, the purpose they serve, their size/performance, and how many users or tasks they support.

Formal / technical definition

Common classification axes are data representation (analog, digital, hybrid), purpose (general-purpose, special-purpose), and capability/scale (micro/personal, workstation, server, mainframe, supercomputer, embedded). These axes are independent: one computer can fit several categories.

Notation and how to read it

By data: analog/digital/hybrid; purpose: general/special; scale/use: personal/workstation/server/mainframe/supercomputer/embedded

How it works

- State the classification axis (purpose, size/capability, or mode of use).
- Describe the defining features of each category.
- Classify a real example only after naming the chosen axis.

Diagram / process map / worked structure

```
Classification is multi-axis: data model + purpose + performance/scale + deployment/use
```

QUICK RECALL

Keep this in mind: Classify an example along two or more named axes instead of treating the categories as one exclusive list.

TOPIC 13 / 48 | Classification of Computers
EXAMPLE, COMMON ERRORS AND EXAM HELP

Classification of Computers by Mode of Use and Features

Worked example / application

Laptop: digital, general-purpose, personal computer. ATM controller: digital, special-purpose embedded system. Supercomputer: digital, high-performance system for large simulations.

Important distinction / practical note

The word 'mode of use' is not perfectly standardized across syllabi; this section covers common textbook axes and states each axis explicitly.

Common mistakes to avoid

- Name the basis of classification before listing types.
- Categories may overlap; a system can be general-purpose and networked.

Quick viva cues

- Core viva answer: Classify an example along two or more named axes instead of treating the categories as one exclusive list.
- Likely follow-up: The word 'mode of use' is not perfectly standardized across syllabi; this section covers common textbook axes and states each axis explicitly.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Classify an example along two or more named axes instead of treating the categories as one exclusive list.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

MODULE 1 COMPLETE

Pause. Review. Then continue.

Module 1 covered what computers do, how the main components work together, and how hardware, software, and computer types fit together.

Before beginning the next module

- Close the notes and explain one important idea from memory.
- Redo one example without looking at its solution.
- Preview the next roadmap to see the order and the ideas you will need.

NEXT MODULE STARTS NEXT

MODULE 2 - Number System, Computer Codes, Operating System

QUICK RECALL

Take a short pause, then begin with the next module's first topic.

Module 2 - what to study and in what order

Module goal

Read and convert positional number systems, understand common character codes, and explain the operating system's role.

Syllabus map - exact section order

- Number System: Number Systems: Introduction and Notation; Positional and Non-Positional Number Systems; Decimal Number System (Base 10); Binary Number System (Base 2); Octal Number System (Base 8); Hexadecimal Number System (Base 16); Decimal Integer to Binary, Octal, or Hexadecimal; Binary, Octal, and Hexadecimal to Decimal; Binary-Octal-Hexadecimal Conversion by Grouping; Conversions with Fractional Parts.
- Computer Codes: Computer Codes: Introduction; BCD (Binary Coded Decimal); EBCDIC; ASCII.
- Operating System: Operating System: Definition and Need; Logical Architecture of an Operating System; Basic Functions of an Operating System; Multiprogramming, Multitasking, and Multithreading.

Recommended study order

- Learn place value and bases before practising conversion algorithms.
- Study BCD, EBCDIC, and ASCII as distinct encodings with different purposes and conventions.
- Finish with OS architecture and its core functions, then compare the three ways of sharing CPU work.

QUICK RECALL

Before moving on, convert one value in each direction and explain why BCD is not ordinary binary.

TOPIC 14 / 48 | Number System
CONCEPT AND DEFINITIONS

Number Systems: Introduction and Notation

Simple definition

A number system is a set of symbols and rules for writing quantities. Computers commonly represent values using binary; people also use decimal, octal, and hexadecimal.

Formal / technical definition

A radix/base b positional numeral system uses b distinct digit values, usually 0 through b-1. A numeral's value is determined by digit coefficients multiplied by powers of the base according to position.

Notation and how to read it

Base b digits: 0,...,b-1; subscript indicates radix, e.g. (1011)2, (11)10, (B)16

How it works

- Identify the base and allowed digit symbols.
- Expand digits using positional weights b^n.
- Check that every digit is smaller than the base.

Diagram / process map / worked structure

```
written numeral -> digit positions -> weighted sum of powers of base -> numeric value
```

QUICK RECALL

Keep this in mind: Expand each digit using its place value and sum to convert to decimal.

TOPIC 14 / 48 | Number System

EXAMPLE, COMMON ERRORS AND EXAM HELP

Number Systems: Introduction and Notation

Worked example / application

(1011)2 = 1x2^3 + 0x2^2 + 1x2^1 + 1x2^0 = 11 decimal.

Important distinction / practical note

A numeral without a stated base can be ambiguous; in computing, write a base subscript or prefix where needed.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Expand each digit using its place value and sum to convert to decimal.
- Likely follow-up: A numeral without a stated base can be ambiguous; in computing, write a base subscript or prefix where needed.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Expand each digit using its place value and sum to convert to decimal.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 15 / 48 | Number System
CONCEPT AND DEFINITIONS

Positional and Non-Positional Number Systems

Simple definition

In a positional system, a symbol's place changes its value. In a non-positional system, a symbol generally has a fixed value and other rules combine symbols.

Formal / technical definition

In positional notation, a digit d_i at position i contributes d_i*b^i. In non-positional notation, symbol values are not determined by powers of a radix; Roman numerals are a familiar mixed additive/subtractive example.

Notation and how to read it

positional value = sum(d_i x b^i); non-positional = symbol values plus convention-specific combination rules

How it works

- Identify the base and allowed digit symbols.
- Expand digits using positional weights b^n.
- Check that every digit is smaller than the base.

Diagram / process map / worked structure

```
(222)10 = 2x10^2 + 2x10^1 + 2x10^0
```

QUICK RECALL

Keep this in mind: Explain how place changes value in positional notation and compare with one non-positional example.

TOPIC 15 / 48 | Number System

EXAMPLE, COMMON ERRORS AND EXAM HELP

Positional and Non-Positional Number Systems

Worked example / application

In 222 decimal, the three 2s mean 200, 20, and 2. In Roman numerals, X denotes ten across its positions, but IX uses a subtractive rule to represent nine.

Important distinction / practical note

Non-positional systems can have ordering/subtraction conventions; do not assume every non-positional notation is simply a sum.

Common mistakes to avoid

- Positional values depend on place and base.
- Roman numerals are a familiar non-positional example; not every written numeral system is positional.

Quick viva cues

- Core viva answer: Explain how place changes value in positional notation and compare with one non-positional example.
- Likely follow-up: Non-positional systems can have ordering/subtraction conventions; do not assume every non-positional notation is simply a sum.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Explain how place changes value in positional notation and compare with one non-positional example.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 16 / 48 | Number System
CONCEPT AND DEFINITIONS

Decimal Number System (Base 10)

Simple definition

Decimal uses ten digits, 0 through 9. Each position is a power of ten.

Formal / technical definition

A base-10 numeral a_n...a_0.a_-1... has value sum(a_i*10^i), with each digit a_i in {0,...,9}.

Notation and how to read it

integer places: ...,10^3,10^2,10^1,10^0; fractional places: 10^-1,10^-2,...

How it works

- Identify the base and allowed digit symbols.
- Expand digits using positional weights b^n.
- Check that every digit is smaller than the base.

Diagram / process map / worked structure

```
5 3 8 . 2 4
100 10 1 0.1 0.01
```

QUICK RECALL

Keep this in mind: Label each place with a power of ten and calculate the weighted sum.

TOPIC 16 / 48 | Number System

EXAMPLE, COMMON ERRORS AND EXAM HELP

Decimal Number System (Base 10)

Worked example / application

(538.24)10 = 5x100 + 3x10 + 8x1 + 2x0.1 + 4x0.01 = 538.24.

Important distinction / practical note

A decimal point separates nonnegative from negative powers; leading/trailing zeros may change formatting without changing value.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Label each place with a power of ten and calculate the weighted sum.
- Likely follow-up: A decimal point separates nonnegative from negative powers; leading/trailing zeros may change formatting without changing value.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Label each place with a power of ten and calculate the weighted sum.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 17 / 48 | Number System

CONCEPT AND DEFINITIONS

Binary Number System (Base 2)

Simple definition

Binary uses only 0 and 1. Each position represents a power of two, which makes it practical for digital circuits with two distinguishable states.

Formal / technical definition

A base-2 numeral is a weighted sum of powers of 2. Binary digits are bits; a group of eight bits is a byte by the usual definition.

Notation and how to read it

binary digits 0,1; place values right-to-left: 1,2,4,8,16,32,...

How it works

- Identify the base and allowed digit symbols.
- Expand digits using positional weights b^n.
- Check that every digit is smaller than the base.

Diagram / process map / worked structure

```
1 1 0 1 0 1
32 16 8 4 2 1
32+16+4+1 = 53
```

QUICK RECALL

Keep this in mind: Write powers of two below the digits, then add only the positions containing 1.

TOPIC 17 / 48 | Number System

EXAMPLE, COMMON ERRORS AND EXAM HELP

Binary Number System (Base 2)

Worked example / application

(110101)2 = 1x32 + 1x16 + 0x8 + 1x4 + 0x2 + 1x1 = (53)10.

Important distinction / practical note

A bit pattern has no inherent meaning until an encoding/type is chosen: the same bits may represent an integer, text, instruction, or part of an image.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Write powers of two below the digits, then add only the positions containing 1.
- Likely follow-up: A bit pattern has no inherent meaning until an encoding/type is chosen: the same bits may represent an integer, text, instruction, or part of an image.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Write powers of two below the digits, then add only the positions containing 1.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 18 / 48 | Number System

CONCEPT AND DEFINITIONS

Octal Number System (Base 8)

Simple definition

Octal uses digits 0 through 7. One octal digit corresponds exactly to a group of three binary bits.

Formal / technical definition

A base-8 numeral uses powers of eight. Because 8=2^3, grouping binary digits from the radix point in groups of three provides direct conversion between binary and octal.

Notation and how to read it

octal digits 0..7; 1 octal digit <-> 3 bits

How it works

- Identify the base and allowed digit symbols.
- Expand digits using positional weights b^n.
- Check that every digit is smaller than the base.

Diagram / process map / worked structure

```
Binary 111010101 -> 111 | 010 | 101 -> octal 7 | 2 | 5
```

QUICK RECALL

Keep this in mind: Use place values for decimal conversion and three-bit grouping for binary conversion.

TOPIC 18 / 48 | Number System

EXAMPLE, COMMON ERRORS AND EXAM HELP

Octal Number System (Base 8)

Worked example / application

(725)8 = 7x64 + 2x8 + 5 = 469 decimal. Binary: 111 010 101 -> (725)8.

Important distinction / practical note

Pad with zeros at the outer end of a group if needed; do not add zeros between existing bits.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Use place values for decimal conversion and three-bit grouping for binary conversion.
- Likely follow-up: Pad with zeros at the outer end of a group if needed; do not add zeros between existing bits.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Use place values for decimal conversion and three-bit grouping for binary conversion.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 19 / 48 | Number System

CONCEPT AND DEFINITIONS

Hexadecimal Number System (Base 16)

Simple definition

Hexadecimal uses 0-9 and A-F for values zero through fifteen. It is a compact way to write binary patterns because one hex digit equals four bits.

Formal / technical definition

A base-16 numeral uses powers of sixteen. Letter digits A-F represent decimal values 10-15. Since 16=2^4, each hexadecimal digit maps to a four-bit binary nibble.

Notation and how to read it

hex digit values: A=10, B=11, C=12, D=13, E=14, F=15; 1 hex digit <-> 4 bits

How it works

- Identify the base and allowed digit symbols.
- Expand digits using positional weights b^n.
- Check that every digit is smaller than the base.

Diagram / process map / worked structure

```
0010 | 1111 -> 2 | F
```

QUICK RECALL

Keep this in mind: Convert each hex digit to its decimal value or four-bit binary group.

TOPIC 19 / 48 | Number System
EXAMPLE, COMMON ERRORS AND EXAM HELP

Hexadecimal Number System (Base 16)

Worked example / application

(2F)16 = 2x16 + 15 = 47 decimal. Binary: 0010 1111 -> (2F)16.

Important distinction / practical note

Hex letters are digits, not variables; note uppercase/lowercase conventions where a programming language matters.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Convert each hex digit to its decimal value or four-bit binary group.
- Likely follow-up: Hex letters are digits, not variables; note uppercase/lowercase conventions where a programming language matters.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Convert each hex digit to its decimal value or four-bit binary group.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 20 / 48 | Number System
CONCEPT AND DEFINITIONS

Decimal Integer to Binary, Octal, or Hexadecimal

Simple definition

Repeatedly divide the decimal integer by the target base. The remainders, read from last to first, form the answer.

Formal / technical definition

For a nonnegative integer N, repeatedly compute quotient q and remainder r such that N=bq+r, with 0<=r<b. Continue with q until zero; reversing the remainder sequence yields the base-b numeral.

Notation and how to read it

N = bq + r; collect r values bottom-to-top

How it works

- Write the source value and target base.
- For an integer, divide repeatedly by the target base and record each remainder.
- Read remainders from last to first; verify by expanding place values.

Diagram / process map / worked structure

```
45 -> r1, r0, r1, r1, r0, r1
reverse -> 101101
```

QUICK RECALL

Keep this in mind: Show every quotient and remainder and verify by converting the result back to decimal.

TOPIC 20 / 48 | Number System

EXAMPLE, COMMON ERRORS AND EXAM HELP

Decimal Integer to Binary, Octal, or Hexadecimal

Worked example / application

```
Convert 45 to each base:
Base 2: 45/2=22 r1; 22/2=11 r0; 11/2=5 r1; 5/2=2 r1; 2/2=1 r0; 1/2=0 r1. Read upward: (101101)2.
Base 8: 45/8=5 r5; 5/8=0 r5. Read upward: (55)8.
Base 16: 45/16=2 r13 (D); 2/16=0 r2. Read upward: (2D)16.
Check: (101101)2=45; (55)8=45; (2D)16=45.
```

Important distinction / practical note

The first remainder is the least-significant digit, so the remainders must be reversed.

Common mistakes to avoid

- Read repeated-division remainders bottom to top.
- A remainder must be less than the target base.

Quick viva cues

- Core viva answer: Show every quotient and remainder and verify by converting the result back to decimal.
- Likely follow-up: The first remainder is the least-significant digit, so the remainders must be reversed.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Show every quotient and remainder and verify by converting the result back to decimal.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 21 / 48 | Number System
CONCEPT AND DEFINITIONS

Binary, Octal, and Hexadecimal to Decimal

Simple definition

Multiply each digit by its place value and add the results. For digits after a radix point, use negative powers of the base.

Formal / technical definition

For numeral (d_n...d_0.d_-1...)_b, value is sum(d_i*b^i). This positional expansion applies to integer and fractional parts.

Notation and how to read it

integer: b^0,b^1,...; fraction: b^-1,b^-2,...

How it works

- Write every digit with its positional weight.
- Multiply each digit by base^position and sum.
- For fractional digits, use negative powers of the base.

Diagram / process map / worked structure

```
3 A . 4 in base 16 -> 3x16^1 + 10x16^0 + 4x16^-1
```

QUICK RECALL

Keep this in mind: Expand the place values, substitute any letter digits, then calculate carefully.

TOPIC 21 / 48 | Number System

EXAMPLE, COMMON ERRORS AND EXAM HELP

Binary, Octal, and Hexadecimal to Decimal

Worked example / application

```
(101101)2 = 32+8+4+1 = 45.
(3A)16 = 3x16 + 10 = 58.
(17.4)8 = 1x8 + 7 + 4/8 = 15.5.
```

Important distinction / practical note

Check that each digit is valid for the stated base before doing arithmetic.

Common mistakes to avoid

- The rightmost integer digit has weight base^0.
- Fractional positions use negative powers; do not start them at zero.

Quick viva cues

- Core viva answer: Expand the place values, substitute any letter digits, then calculate carefully.
- Likely follow-up: Check that each digit is valid for the stated base before doing arithmetic.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Expand the place values, substitute any letter digits, then calculate carefully.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 22 / 48 | Number System
CONCEPT AND DEFINITIONS

Binary-Octal-Hexadecimal Conversion by Grouping

Simple definition

Because octal is 2^3 and hexadecimal is 2^4, convert binary by grouping bits rather than doing long division.

Formal / technical definition

For binary-to-octal, partition digits into groups of three from the radix point outward; for binary-to-hex, use groups of four. For reverse conversion, replace each octal digit by three bits and each hex digit by four bits.

Notation and how to read it

octal map: 0..7 -> 000..111; hex map: 0..F -> 0000..1111

How it works

- Binary to octal: group bits in threes from the radix point outward.
- Binary to hexadecimal: group bits in fours; pad only with leading/trailing zeros as needed.
- Convert each group independently and preserve the radix point.

Diagram / process map / worked structure

```
Binary integer grouping starts from the right; fractional grouping starts from the radix point toward the right
```

QUICK RECALL

Keep this in mind: Mark groups with separators, convert each group, and reassemble the digits.

TOPIC 22 / 48 | Number System

EXAMPLE, COMMON ERRORS AND EXAM HELP

Binary-Octal-Hexadecimal Conversion by Grouping

Worked example / application

```
(11010110)2 -> 011 010 110 -> (326)8.
(11010110)2 -> 1101 0110 -> (D6)16.
```

Important distinction / practical note

Pad only the outermost group with zeros; the represented value remains unchanged.

Common mistakes to avoid

- Group from the radix point outward, not from the left edge in all cases.
- Pad with zeros only; do not change the value.

Quick viva cues

- Core viva answer: Mark groups with separators, convert each group, and reassemble the digits.
- Likely follow-up: Pad only the outermost group with zeros; the represented value remains unchanged.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Mark groups with separators, convert each group, and reassemble the digits.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 23 / 48 | Number System

CONCEPT AND DEFINITIONS

Conversions with Fractional Parts

Simple definition

To convert a decimal fraction to another base, multiply the fraction repeatedly by the target base; the whole-number parts become digits from left to right.

Formal / technical definition

For 0<=f<1, compute f*b; record floor(f*b) as the next digit, then repeat with the remaining fractional part. The process may terminate or repeat; some fractions have infinite expansions in the target base.

Notation and how to read it

fraction -> multiply by base -> take integer digit -> continue with fractional remainder

How it works

- For decimal fraction to base b, repeatedly multiply the fractional part by b.
- Record each integer part in order; continue with the new fractional part.
- Stop when exact, when the desired precision is reached, or when a pattern repeats.

Diagram / process map / worked structure

```
0.625 -> 1.25 (digit1), .25 -> .5 (digit0), .5 -> 1.0 (digit1)
```

QUICK RECALL

Keep this in mind: Keep digits in the order produced and track the remaining fraction after each multiplication.

TOPIC 23 / 48 | Number System

EXAMPLE, COMMON ERRORS AND EXAM HELP

Conversions with Fractional Parts

Worked example / application

Exact example: 0.625x2=1.25 -> digit 1; 0.25x2=0.5 -> 0; 0.5x2=1.0 -> 1. So (0.625)10=(0.101)2.

Repeating example: 0.1x2=0.2 -> 0; 0.2x2=0.4 -> 0; 0.4x2=0.8 -> 0; 0.8x2=1.6 -> 1; 0.6x2=1.2 -> 1; remainder 0.2 repeats. Therefore (0.1)10=(0.000110011...)2; first five bits .00011 are approximate.

Important distinction / practical note

Stop at the requested precision if the expansion repeats; state whether the result is exact or approximate.

Common mistakes to avoid

- Fractional multiplication digits are read in the order produced.
- Some fractions repeat forever in the target base; state the chosen precision.

Quick viva cues

- Core viva answer: Keep digits in the order produced and track the remaining fraction after each multiplication.
- Likely follow-up: Stop at the requested precision if the expansion repeats; state whether the result is exact or approximate.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Keep digits in the order produced and track the remaining fraction after each multiplication.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 24 / 48 | Computer Codes
CONCEPT AND DEFINITIONS

Computer Codes: Introduction

Simple definition

A computer code assigns bit patterns to symbols or values so that data can be stored, processed, and exchanged consistently.

Formal / technical definition

An encoding maps a set of symbols or numeric values to code words. A code must be interpreted using the same convention at both ends; different encodings may use different lengths, ranges, and error/control capabilities.

Notation and how to read it

symbol/value -> encoding rule -> bit pattern; decoder applies the same rule in reverse

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
text/data -> encoder -> bits -> storage/transmission -> decoder -> interpreted value
```

QUICK RECALL

Keep this in mind: Identify what is being encoded: decimal digit, character, instruction, or control signal.

TOPIC 24 / 48 | Computer Codes

EXAMPLE, COMMON ERRORS AND EXAM HELP

Computer Codes: Introduction

Worked example / application

A character 'A' may be encoded as 01000001 in ASCII-compatible encoding. A decimal digit 7 encoded in BCD uses 0111 for the digit, not the ordinary 4-bit binary representation of the entire multi-digit number.

Important distinction / practical note

Do not confuse a character code (ASCII/EBCDIC) with a numeric representation (binary/BCD).

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Identify what is being encoded: decimal digit, character, instruction, or control signal.
- Likely follow-up: Do not confuse a character code (ASCII/EBCDIC) with a numeric representation (binary/BCD).

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Identify what is being encoded: decimal digit, character, instruction, or control signal.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 25 / 48 | Computer Codes
CONCEPT AND DEFINITIONS

BCD (Binary Coded Decimal)

Simple definition

BCD encodes each decimal digit separately using a four-bit pattern. It keeps decimal digits easy to read, but uses more bits than compact binary for many values.

Formal / technical definition

In 8421 BCD, each decimal digit d in 0..9 is represented by its 4-bit binary code. A multi-digit decimal numeral is represented by concatenating digit codes; patterns 1010-1111 are invalid for a single 8421 BCD digit.

Notation and how to read it

8421 weights per digit: 8,4,2,1; encode each decimal digit separately

How it works

- Separate the decimal number into individual decimal digits.
- Encode each digit using its four-bit 8421 group.
- Keep the groups separate; do not treat the full bit string as an ordinary binary integer.

Diagram / process map / worked structure

```
Decimal 5 | 9 -> BCD 0101 | 1001
BCD 0101 1001 -> decimal digits 59
```

QUICK RECALL

Keep this in mind: Split the decimal value into digits, encode each digit in four bits, and preserve leading zero bits in each nibble.

TOPIC 25 / 48 | Computer Codes

EXAMPLE, COMMON ERRORS AND EXAM HELP

BCD (Binary Coded Decimal)

Worked example / application

Decimal 59 -> 5=0101 and 9=1001, so BCD is 0101 1001. Ordinary binary 59 is 111011; these representations are not the same.

Important distinction / practical note

BCD is not just writing the whole decimal number in binary. For 59, BCD uses 8 bits while ordinary binary uses 6 bits.

Common mistakes to avoid

- BCD encodes each decimal digit separately; it is not ordinary binary for the whole number.
- Valid 8421 BCD nibbles are 0000 through 1001; 1010 through 1111 are not decimal digits.

Quick viva cues

- Core viva answer: Split the decimal value into digits, encode each digit in four bits, and preserve leading zero bits in each nibble.
- Likely follow-up: BCD is not just writing the whole decimal number in binary. For 59, BCD uses 8 bits while ordinary binary uses 6 bits.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Split the decimal value into digits, encode each digit in four bits, and preserve leading zero bits in each nibble.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 26 / 48 | Computer Codes
CONCEPT AND DEFINITIONS

EBCDIC

Simple definition

EBCDIC is an 8-bit character encoding developed for IBM systems. It assigns codes to letters, digits, punctuation, and control characters.

Formal / technical definition

Extended Binary Coded Decimal Interchange Code maps characters to 8-bit code values and has multiple code pages/variants. It is not byte-for-byte compatible with ASCII; character ordering and exact mappings depend on the EBCDIC code page.

Notation and how to read it

8-bit code value -> character under a specific EBCDIC code page

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
bytes + selected code page -> decoded text; wrong code page -> mojibake/incorrect symbols
```

QUICK RECALL

Keep this in mind: Expand the acronym and explain why decoding must use the matching code page.

TOPIC 26 / 48 | Computer Codes

EXAMPLE, COMMON ERRORS AND EXAM HELP

EBCDIC

Worked example / application

A text file encoded in one EBCDIC code page can display incorrectly if decoded as ASCII/UTF-8 because the same bytes have different meanings.

Important distinction / practical note

Do not assume one universal EBCDIC character table; identify the code page when exact byte values matter.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Expand the acronym and explain why decoding must use the matching code page.
- Likely follow-up: Do not assume one universal EBCDIC character table; identify the code page when exact byte values matter.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Expand the acronym and explain why decoding must use the matching code page.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 27 / 48 | Computer Codes
CONCEPT AND DEFINITIONS

ASCII

Simple definition

ASCII is a character code for basic English letters, digits, punctuation, and control characters. Its original code values use seven bits.

Formal / technical definition

The ASCII standard defines 128 code positions (0-127): 33 control characters plus 95 printable characters. Eight-bit bytes may carry ASCII-compatible values in their lower seven bits, while extended encodings are not all standard ASCII.

Notation and how to read it

ASCII range 0..127; printable characters mostly 32..126; '0'..'9' = 48..57

How it works

- Identify whether a question means 7-bit ASCII or an 8-bit extension.
- Map each character to its numeric code, then binary if requested.
- For a string, encode each character separately in order.

Diagram / process map / worked structure

```
character -> code point/value -> binary byte pattern -> decoder maps it back
```

QUICK RECALL

Keep this in mind: State ASCII's 7-bit range and distinguish it from Unicode and vendor-specific extended code pages.

TOPIC 27 / 48 | Computer Codes
EXAMPLE, COMMON ERRORS AND EXAM HELP

ASCII

Worked example / application

ASCII 'A'=65 decimal=0x41=01000001 in an 8-bit display. Lowercase 'a'=97 decimal=0x61.

Important distinction / practical note

ASCII is not a full Unicode encoding. UTF-8 preserves ASCII byte values for the basic range but represents many other characters with multiple bytes.

Common mistakes to avoid

- Standard ASCII is 7-bit; extended 8-bit encodings are not one universal ASCII standard.
- A character is not necessarily one byte in every modern text encoding.

Quick viva cues

- Core viva answer: State ASCII's 7-bit range and distinguish it from Unicode and vendor-specific extended code pages.
- Likely follow-up: ASCII is not a full Unicode encoding. UTF-8 preserves ASCII byte values for the basic range but represents many other characters with multiple bytes.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: State ASCII's 7-bit range and distinguish it from Unicode and vendor-specific extended code pages.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 28 / 48 | Operating System

CONCEPT AND DEFINITIONS

Operating System: Definition and Need

Simple definition

An operating system is the core software that manages computer resources and provides services so applications and users can use hardware conveniently.

Formal / technical definition

An OS is system software that allocates CPU, memory, storage, and I/O resources; controls execution; enforces protection; and exposes abstractions such as processes, files, and devices.

Notation and how to read it

user/application -> OS services -> hardware; OS manages resources and protects execution

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
Applications -> system call/API -> OS kernel/services -> device drivers/controllers -> hardware
```

QUICK RECALL

Keep this in mind: Define the OS, name its resource-management role, and show one service request path.

TOPIC 28 / 48 | Operating System
EXAMPLE, COMMON ERRORS AND EXAM HELP

Operating System: Definition and Need

Worked example / application

When a program saves a file, it requests a file operation from the OS; the OS checks access, coordinates storage, and reports success/failure.

Important distinction / practical note

The OS is not only a graphical interface; a GUI is one way to interact with services, while many OS functions work without a GUI.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Define the OS, name its resource-management role, and show one service request path.
- Likely follow-up: The OS is not only a graphical interface; a GUI is one way to interact with services, while many OS functions work without a GUI.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Define the OS, name its resource-management role, and show one service request path.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 29 / 48 | Operating System
CONCEPT AND DEFINITIONS

Logical Architecture of an Operating System

Simple definition

A useful OS picture has users and applications at the top, OS services/kernel in the middle, and hardware at the bottom.

Formal / technical definition

A logical view separates user mode from privileged kernel mode. Applications call APIs/system-call interfaces; kernel subsystems manage processes, memory, files, I/O, and protection; drivers communicate with device controllers.

Notation and how to read it

user mode | API/system call boundary | kernel mode | drivers | hardware

How it works

- Start at the user and application level.
- Trace a request through the interface/system-call boundary to the kernel.
- Show how kernel managers coordinate hardware and return a result.

Diagram / process map / worked structure

```
User & applications -> libraries/API -> system calls -> kernel subsystems -> drivers -> devices
```

QUICK RECALL

Keep this in mind: Draw the privilege boundary and trace a request from application to device and back.

TOPIC 29 / 48 | Operating System
EXAMPLE, COMMON ERRORS AND EXAM HELP

Logical Architecture of an Operating System

Worked example / application

A browser asks to read a file through an API; the system-call boundary enters kernel services, file-system code resolves storage blocks, and a driver/device retrieves them.

Important distinction / practical note

Actual OS designs may be monolithic, layered, microkernel, modular, or hybrid; the logical service path does not imply one fixed internal structure.

Common mistakes to avoid

- The OS is not a single user application; privileged kernel work is distinct.
- A graphical interface is one way to interact with an OS, not the whole OS.

Quick viva cues

- Core viva answer: Draw the privilege boundary and trace a request from application to device and back.
- Likely follow-up: Actual OS designs may be monolithic, layered, microkernel, modular, or hybrid; the logical service path does not imply one fixed internal structure.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Draw the privilege boundary and trace a request from application to device and back.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 30 / 48 | Operating System

CONCEPT AND DEFINITIONS

Basic Functions of an Operating System

Simple definition

An OS runs programs, shares resources, stores files, communicates with devices, and protects the system from errors or unauthorized use.

Formal / technical definition

Core OS responsibilities include process/thread management, CPU scheduling, memory management, file-system management, I/O/device management, secondary-storage management, networking, protection/security, error handling, accounting, and user interfaces.

Notation and how to read it

resource management + abstractions + protection + user/application interface

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
program execution | scheduling | memory | files | I/O | security | networking | errors
```

QUICK RECALL

Keep this in mind: Choose four OS functions and explain their resource, action, and user-visible benefit.

TOPIC 30 / 48 | Operating System
EXAMPLE, COMMON ERRORS AND EXAM HELP

Basic Functions of an Operating System

Worked example / application

If two applications need CPU time, the scheduler shares the processor; if one reads a file, file and device services coordinate access; if memory is limited, memory management tracks allocation and protection.

Important distinction / practical note

Not every OS provides the same feature set; embedded and real-time systems may omit or constrain services.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Choose four OS functions and explain their resource, action, and user-visible benefit.
- Likely follow-up: Not every OS provides the same feature set; embedded and real-time systems may omit or constrain services.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Choose four OS functions and explain their resource, action, and user-visible benefit.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 31 / 48 | Operating System
CONCEPT AND DEFINITIONS

Multiprogramming, Multitasking, and Multithreading

Simple definition

These terms describe ways an operating system keeps work moving: several programs in memory, several tasks sharing time, or several execution paths inside one program.

Formal / technical definition

Multiprogramming keeps multiple jobs in memory and switches CPU to another when a job waits, improving CPU utilization. Multitasking/time-sharing rapidly allocates CPU slices to tasks for responsiveness. Multithreading allows multiple threads within a process to execute concurrently while sharing process resources.

Notation and how to read it

multiprogramming: jobs in memory; multitasking: time-shared tasks; multithreading: execution paths within a process

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
one CPU: A runs -> waits -> B runs -> timer switch -> A/B resume; multiple cores may execute threads in parallel
```

QUICK RECALL

Keep this in mind: Compare unit of work, memory relationship, switching goal, and a concrete example.

TOPIC 31 / 48 | Operating System

EXAMPLE, COMMON ERRORS AND EXAM HELP

Multiprogramming, Multitasking, and Multithreading

Worked example / application

While Job A waits for disk, OS runs Job B (multiprogramming). A desktop switches among editor and browser time slices (multitasking). A browser process may use separate threads for UI and work (multithreading).

Important distinction / practical note

Concurrency means overlapping progress; parallelism means simultaneous execution. They are related but not identical.

Common mistakes to avoid

- Multiprogramming keeps work ready to use CPU time; multitasking emphasizes responsive sharing.
- A thread is an execution path within a process; threads share some process resources.

Quick viva cues

- Core viva answer: Compare unit of work, memory relationship, switching goal, and a concrete example.
- Likely follow-up: Concurrency means overlapping progress; parallelism means simultaneous execution. They are related but not identical.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Compare unit of work, memory relationship, switching goal, and a concrete example.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

MODULE 2 COMPLETE

Pause. Review. Then continue.

Module 2 covered number representation and conversion, character codes, and how operating systems organize computer resources.

Before beginning the next module

- Close the notes and explain one important idea from memory.
- Redo one example without looking at its solution.
- Preview the next roadmap to see the order and the ideas you will need.

NEXT MODULE STARTS NEXT

MODULE 3 - Algorithms, Internet, MS-Office, Computer Languages

QUICK RECALL

Take a short pause, then begin with the next module's first topic.

Module 3 - what to study and in what order

Module goal

Write clear algorithms, understand the Internet and Web, identify Office tools, and compare computer languages and translators.

Syllabus map - exact section order

- Algorithms: Algorithms: Introduction and Definition; Algorithm Criteria and Notation; Types of Algorithms.
- Internet: Internet: Introduction; Evolution of the Internet; Basic Services of the Internet; World Wide Web (WWW); Web Browsers; Uses of the Internet.
- MS-Office: Design Goals of MS-Office; MS-Word; MS-PowerPoint; MS-Excel; MS-Access.
- Computer Languages: Computer Languages: Introduction and Classification; Advantages and Limitations of Language Levels; Language Translators.

Recommended study order

- Learn what makes an algorithm precise and how to represent one.
- Trace Internet history and services; keep the Internet distinct from the Web and browser tools.
- Compare the four Office components, then study language levels and translation paths.

QUICK RECALL

Before moving on, explain one algorithm, one Internet service, an Office tool choice, and a translator path.

TOPIC 32 / 48 | Algorithms
CONCEPT AND DEFINITIONS

Algorithms: Introduction and Definition

Simple definition

An algorithm is a clear, finite set of steps for solving a problem or producing a result.

Formal / technical definition

An algorithm is a well-defined sequence of effective, unambiguous instructions that maps allowed inputs to outputs and terminates for inputs in its stated domain (for a terminating algorithm). It is independent of a particular programming language.

Notation and how to read it

problem + valid input -> finite precise steps -> required output

How it works

- State inputs, outputs, and assumptions.
- Write finite, unambiguous steps in the required notation.
- Trace a small case and check termination and correctness.

Diagram / process map / worked structure

```
Input -> algorithm steps -> output; implementation can be written in several programming languages
```

QUICK RECALL

Keep this in mind: State input, steps, output, and termination behavior for a small problem.

TOPIC 32 / 48 | Algorithms
EXAMPLE, COMMON ERRORS AND EXAM HELP

Algorithms: Introduction and Definition

Worked example / application

Find the larger of two numbers: read A,B; if A>B output A; otherwise output B. The algorithm includes behavior for equality too.

Important distinction / practical note

An algorithm is not the same as a program: a program expresses an algorithm in a particular language and environment.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: State input, steps, output, and termination behavior for a small problem.
- Likely follow-up: An algorithm is not the same as a program: a program expresses an algorithm in a particular language and environment.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: State input, steps, output, and termination behavior for a small problem.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 33 / 48 | Algorithms

CONCEPT AND DEFINITIONS

Algorithm Criteria and Notation

Simple definition

A good algorithm says exactly what to do, works for the allowed inputs, finishes, and can be carried out step by step.

Formal / technical definition

Common criteria are input, output, definiteness, finiteness, effectiveness, and correctness. Algorithms may be written in plain structured English, pseudocode, flowcharts, or a programming language; consistent notation matters more than decorative symbols.

Notation and how to read it

READ/INPUT; SET/assignment; IF...THEN...ELSE; WHILE/FOR; OUTPUT/PRINT; <- means assign

How it works

- State inputs, outputs, and assumptions.
- Write finite, unambiguous steps in the required notation.
- Trace a small case and check termination and correctness.

Diagram / process map / worked structure

```
Start -> initialize -> test -> body/update -> repeat -> output -> stop
```

QUICK RECALL

Keep this in mind: List the criteria and annotate a pseudocode example with input, output, condition, and termination.

TOPIC 33 / 48 | Algorithms
EXAMPLE, COMMON ERRORS AND EXAM HELP

Algorithm Criteria and Notation

Worked example / application

Pseudocode for sum 1..n: READ n; total <- 0; FOR i <- 1 TO n DO total <- total+i; OUTPUT total. Preconditions should say n is a nonnegative integer.

Important distinction / practical note

Pseudocode is language-independent but must still define boundaries, loop updates, and behavior for invalid inputs.

Common mistakes to avoid

- A finite algorithm must make progress toward stopping.
- Pseudocode has no single universal punctuation standard; clarity and consistency matter.

Quick viva cues

- Core viva answer: List the criteria and annotate a pseudocode example with input, output, condition, and termination.
- Likely follow-up: Pseudocode is language-independent but must still define boundaries, loop updates, and behavior for invalid inputs.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: List the criteria and annotate a pseudocode example with input, output, condition, and termination.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 34 / 48 | Algorithms
CONCEPT AND DEFINITIONS

Types of Algorithms

Simple definition

Algorithms can be grouped by the strategy they use, such as trying all options, dividing a problem, using a greedy choice, or repeating improvements.

Formal / technical definition

Common strategy families include brute force, divide-and-conquer, greedy, dynamic programming, backtracking, recursion/iteration, and randomized methods. These categories describe design approaches and may overlap; they are broader than this introductory syllabus's specific algorithm examples.

Notation and how to read it

strategy family = recurring method for structuring steps; choose by problem constraints and correctness proof

How it works

- State inputs, outputs, and assumptions.
- Write finite, unambiguous steps in the required notation.
- Trace a small case and check termination and correctness.

Diagram / process map / worked structure

```
problem -> choose strategy -> specify steps -> prove correctness -> analyze resources
```

QUICK RECALL

Keep this in mind: For each strategy, explain its core idea and one example; do not memorize names without the procedure.

TOPIC 34 / 48 | Algorithms

EXAMPLE, COMMON ERRORS AND EXAM HELP

Types of Algorithms

Worked example / application

Linear search is a simple sequential strategy; binary search repeatedly halves a sorted search interval; an algorithm for a maze may use backtracking to explore choices and undo dead ends.

Important distinction / practical note

This syllabus lists 'Types' without naming a taxonomy; the notes use common introductory strategy families and label them as classification vocabulary.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: For each strategy, explain its core idea and one example; do not memorize names without the procedure.
- Likely follow-up: This syllabus lists 'Types' without naming a taxonomy; the notes use common introductory strategy families and label them as classification vocabulary.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: For each strategy, explain its core idea and one example; do not memorize names without the procedure.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 35 / 48 | Internet

CONCEPT AND DEFINITIONS

Internet: Introduction

Simple definition

The Internet is a worldwide network of connected networks that exchange data using agreed communication protocols.

Formal / technical definition

The Internet is an interconnected system of packet-switched networks using the Internet Protocol suite. Routers forward packets between networks; application protocols such as HTTP, email protocols, and DNS support particular services.

Notation and how to read it

device -> local network -> ISP/network -> routers -> destination network/server

How it works

- Identify the user's task and the relevant Internet service.
- Trace client request, network delivery, service response, and user-visible result.
- Keep the Internet (network infrastructure) distinct from the Web (one service).

Diagram / process map / worked structure

```
Application data -> TCP/UDP and IP layers -> packets -> links/routers -> destination reassembly/application
```

QUICK RECALL

Keep this in mind: Define the Internet and distinguish networks/protocols from an application service.

TOPIC 35 / 48 | Internet

EXAMPLE, COMMON ERRORS AND EXAM HELP

Internet: Introduction

Worked example / application

A browser sends an HTTP request through a local network and ISP; packets cross routers to a server, and response packets return to the device.

Important distinction / practical note

The Internet is the underlying network infrastructure; the Web is one service that uses it.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Define the Internet and distinguish networks/protocols from an application service.
- Likely follow-up: The Internet is the underlying network infrastructure; the Web is one service that uses it.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Define the Internet and distinguish networks/protocols from an application service.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 36 / 48 | Internet

CONCEPT AND DEFINITIONS

Evolution of the Internet

Simple definition

The Internet grew from research networking into a global infrastructure for communication, information, commerce, and digital services.

Formal / technical definition

Key milestones include ARPANET's first host connections in 1969, development of internetworking and TCP/IP, the ARPANET transition to TCP/IP on 1 January 1983, expansion through research/commercial networks, and broad public growth in the 1990s. The Web was proposed separately in 1989 and became a major Internet service.

Notation and how to read it

Internet = network-of-networks + protocols; Web = linked information system running over the Internet

How it works

- Identify the user's task and the relevant Internet service.
- Trace client request, network delivery, service response, and user-visible result.
- Keep the Internet (network infrastructure) distinct from the Web (one service).

Diagram / process map / worked structure

```
ARPANET/research networks -> TCP/IP internetworking -> global Internet -> Web and many additional services
```

QUICK RECALL

Keep this in mind: Place three milestones in order and state one difference between Internet and Web.

TOPIC 36 / 48 | Internet

EXAMPLE, COMMON ERRORS AND EXAM HELP

Evolution of the Internet

Worked example / application

Timeline: packet-switching research -> ARPANET (1969) -> TCP/IP standard transition (1983) -> research/commercial expansion -> Web proposal (1989), first browser/server (1990) -> public Web growth.

Important distinction / practical note

Do not equate the Internet's creation with the invention of the Web; the Web came later and is only one Internet service.

Common mistakes to avoid

- The Internet and the Web are not synonyms.
- The Internet developed through many contributors and networks, not a single invention date.

Quick viva cues

- Core viva answer: Place three milestones in order and state one difference between Internet and Web.
- Likely follow-up: Do not equate the Internet's creation with the invention of the Web; the Web came later and is only one Internet service.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Place three milestones in order and state one difference between Internet and Web.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 37 / 48 | Internet

CONCEPT AND DEFINITIONS

Basic Services of the Internet

Simple definition

Internet services are the practical activities people use networks for: messaging, file transfer, web access, calls, remote access, and collaboration.

Formal / technical definition

Internet services are application-level functions built over network protocols. Examples include email, web browsing, file transfer, remote login/administration, messaging/voice/video, cloud services, and information retrieval; each uses appropriate protocols and client/server or peer-to-peer designs.

Notation and how to read it

service = client/application + protocol(s) + server/peer + network transport

How it works

- Identify the user's task and the relevant Internet service.
- Trace client request, network delivery, service response, and user-visible result.
- Keep the Internet (network infrastructure) distinct from the Web (one service).

Diagram / process map / worked structure

```
user task -> client app -> application protocol -> network -> remote service -> response
```

QUICK RECALL

Keep this in mind: Name four services and describe the user task and direction/type of data for each.

TOPIC 37 / 48 | Internet

EXAMPLE, COMMON ERRORS AND EXAM HELP

Basic Services of the Internet

Worked example / application

Email sends store-and-forward messages; web browsing retrieves linked resources; video calls exchange time-sensitive audio/video; file transfer moves stored data between systems.

Important distinction / practical note

Services can use several underlying protocols; one service name does not imply one single network protocol.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Name four services and describe the user task and direction/type of data for each.
- Likely follow-up: Services can use several underlying protocols; one service name does not imply one single network protocol.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Name four services and describe the user task and direction/type of data for each.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 38 / 48 | Internet

CONCEPT AND DEFINITIONS

World Wide Web (WWW)

Simple definition

The Web is a system of linked pages and resources identified by web addresses and accessed through browsers using web protocols.

Formal / technical definition

The World Wide Web is a distributed hypertext/information system based on resources identified by URIs/URLs, transferred using HTTP or HTTPS, and commonly represented with HTML plus related media/style/scripts. A website may contain many resources across servers.

Notation and how to read it

URL/URI identifies resource; HTTP(S) transfers; HTML structures content; browser interprets/renders

How it works

- Identify the user's task and the relevant Internet service.
- Trace client request, network delivery, service response, and user-visible result.
- Keep the Internet (network infrastructure) distinct from the Web (one service).

Diagram / process map / worked structure

```
URL -> DNS lookup -> HTTP(S) request -> server response -> browser rendering -> hyperlinks to other resources
```

QUICK RECALL

Keep this in mind: Trace what happens from entering a web address to seeing a rendered page.

TOPIC 38 / 48 | Internet

EXAMPLE, COMMON ERRORS AND EXAM HELP

World Wide Web (WWW)

Worked example / application

Enter a URL -> browser resolves the host using DNS -> requests a resource over HTTP(S) -> server returns HTML/images -> browser renders and follows links.

Important distinction / practical note

The Web is not the Internet itself; email, DNS, and many other services also use the Internet.

Common mistakes to avoid

- The Web is a service built on the Internet; email is another service.
- A browser is a client program; a URL identifies a resource location.

Quick viva cues

- Core viva answer: Trace what happens from entering a web address to seeing a rendered page.
- Likely follow-up: The Web is not the Internet itself; email, DNS, and many other services also use the Internet.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Trace what happens from entering a web address to seeing a rendered page.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 39 / 48 | Internet

CONCEPT AND DEFINITIONS

Web Browsers

Simple definition

A browser is client software for requesting, receiving, interpreting, and displaying Web content.

Formal / technical definition

A browser manages URLs, network requests, HTML/CSS/JavaScript processing, rendering, user interaction, security boundaries, storage/cache, and downloads. Browser implementations differ in engines and features.

Notation and how to read it

browser functions: navigation | request/response | rendering | scripts | cache/cookies | security controls

How it works

- Identify the user's task and the relevant Internet service.
- Trace client request, network delivery, service response, and user-visible result.
- Keep the Internet (network infrastructure) distinct from the Web (one service).

Diagram / process map / worked structure

```
URL input -> network fetch -> parse resource -> layout/render -> interaction/navigation
```

QUICK RECALL

Keep this in mind: Compare browser and search engine, then explain the browser's request-to-render path.

TOPIC 39 / 48 | Internet

EXAMPLE, COMMON ERRORS AND EXAM HELP

Web Browsers

Worked example / application

A browser requests a page, parses HTML into a document structure, applies CSS for presentation, executes allowed scripts, and displays the result.

Important distinction / practical note

A browser is not a search engine; a search engine is a service/site accessed through a browser.

Common mistakes to avoid

- A browser and a search engine are different things.
- Web pages may include several resources, not just one HTML file.

Quick viva cues

- Core viva answer: Compare browser and search engine, then explain the browser's request-to-render path.
- Likely follow-up: A browser is not a search engine; a search engine is a service/site accessed through a browser.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Compare browser and search engine, then explain the browser's request-to-render path.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 40 / 48 | Internet

CONCEPT AND DEFINITIONS

Uses of the Internet

Simple definition

People use the Internet to communicate, learn, work, transact, create, access services, and share information.

Formal / technical definition

Internet use spans communication, information retrieval, education, collaboration, commerce, banking, entertainment, cloud computing, remote work, public services, and device connectivity. Responsible use includes security, privacy, source evaluation, and accessibility considerations.

Notation and how to read it

use case = user need + Internet service + data exchanged + benefit/risks

How it works

- Identify the user's task and the relevant Internet service.
- Trace client request, network delivery, service response, and user-visible result.
- Keep the Internet (network infrastructure) distinct from the Web (one service).

Diagram / process map / worked structure

```
user -> application/service -> network -> remote information/person/system -> response
```

QUICK RECALL

Keep this in mind: Explain two uses with benefit and one responsible-use concern for each.

TOPIC 40 / 48 | Internet

EXAMPLE, COMMON ERRORS AND EXAM HELP

Uses of the Internet

Worked example / application

For online learning: a learner accesses a course page, streams a lesson, submits work, and receives feedback through networked applications.

Important distinction / practical note

Benefits depend on access, reliable connectivity, digital literacy, privacy safeguards, and trustworthy information.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Explain two uses with benefit and one responsible-use concern for each.
- Likely follow-up: Benefits depend on access, reliable connectivity, digital literacy, privacy safeguards, and trustworthy information.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Explain two uses with benefit and one responsible-use concern for each.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 41 / 48 | MS-Office

CONCEPT AND DEFINITIONS

Design Goals of MS-Office

Simple definition

Office productivity suites aim to help users create, organize, calculate, present, and share work using related applications.

Formal / technical definition

A productivity suite groups applications with consistent interaction patterns, document/file handling, editing, formatting, collaboration, and data exchange. Exact features vary by version and licensing; the syllabus refers to the Microsoft Office family.

Notation and how to read it

suite goals: productivity | consistency | integration | document creation | data analysis | presentation | sharing

How it works

- Identify whether the task is documents, slides, calculations, or structured records.
- Choose the matching component and its main data object.
- Describe the output that helps the user complete the task.

Diagram / process map / worked structure

```
data/ideas -> appropriate Office component -> document/workbook/presentation/database -> review/share
```

QUICK RECALL

Keep this in mind: Map each design goal to one application action and one user benefit.

TOPIC 41 / 48 | MS-Office

EXAMPLE, COMMON ERRORS AND EXAM HELP

Design Goals of MS-Office

Worked example / application

A student can draft a report in Word, summarize marks in Excel, present findings in PowerPoint, and organize structured records in Access.

Important distinction / practical note

Do not assume every Office version includes the same apps or that files behave identically across software versions.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Map each design goal to one application action and one user benefit.
- Likely follow-up: Do not assume every Office version includes the same apps or that files behave identically across software versions.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Map each design goal to one application action and one user benefit.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 42 / 48 | MS-Office

CONCEPT AND DEFINITIONS

MS-Word

Simple definition

Word is a word-processing application for creating, editing, formatting, reviewing, and printing text-centered documents.

Formal / technical definition

A word processor manages document structure and presentation, including paragraphs, styles, page layout, tables, references, comments, spelling tools, and export/print functions.

Notation and how to read it

Word tasks: create/edit text; format with styles; page layout; tables/images; review; print/export

How it works

- Identify whether the task is documents, slides, calculations, or structured records.
- Choose the matching component and its main data object.
- Describe the output that helps the user complete the task.

Diagram / process map / worked structure

```
content -> structure with headings/styles -> layout -> review -> print/share/export
```

QUICK RECALL

Keep this in mind: Name three Word tasks and explain why styles improve document consistency.

TOPIC 42 / 48 | MS-Office

EXAMPLE, COMMON ERRORS AND EXAM HELP

MS-Word

Worked example / application

Use heading styles for sections, insert page numbers, add a table, run spelling checks, and export a report to PDF.

Important distinction / practical note

Styles are more reliable than manual formatting for consistent headings and automatic navigation/table of contents.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Name three Word tasks and explain why styles improve document consistency.
- Likely follow-up: Styles are more reliable than manual formatting for consistent headings and automatic navigation/table of contents.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Name three Word tasks and explain why styles improve document consistency.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 43 / 48 | MS-Office

CONCEPT AND DEFINITIONS

MS-PowerPoint

Simple definition

PowerPoint is presentation software for organizing a message into slides that combine text, visuals, data, and speaker delivery.

Formal / technical definition

Presentation software creates a sequence of slides with layouts, themes, objects, transitions, animations, presenter notes, and delivery/export modes. Effective slides support a spoken explanation rather than duplicating a full essay.

Notation and how to read it

presentation = audience + purpose + ordered slides + visual evidence + delivery

How it works

- Identify whether the task is documents, slides, calculations, or structured records.
- Choose the matching component and its main data object.
- Describe the output that helps the user complete the task.

Diagram / process map / worked structure

```
message outline -> slide structure -> text/visuals -> rehearsal -> presentation/export
```

QUICK RECALL

Keep this in mind: Describe slide design choices that make a presentation clear and accessible.

TOPIC 43 / 48 | MS-Office

EXAMPLE, COMMON ERRORS AND EXAM HELP

MS-PowerPoint

Worked example / application

A short presentation can use a title slide, problem, three evidence slides with charts/images, conclusion, and references.

Important distinction / practical note

Too much text, inconsistent layout, and decorative effects can distract from the central message.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Describe slide design choices that make a presentation clear and accessible.
- Likely follow-up: Too much text, inconsistent layout, and decorative effects can distract from the central message.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Describe slide design choices that make a presentation clear and accessible.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 44 / 48 | MS-Office

CONCEPT AND DEFINITIONS

MS-Excel

Simple definition

Excel is spreadsheet software for storing values in cells, calculating with formulas, summarizing data, and making charts.

Formal / technical definition

A spreadsheet organizes data in rows and columns. Cells can contain constants, formulas, or functions; references may be relative, absolute, or mixed. Tables, sorting/filtering, charts, and analysis tools support structured data work.

Notation and how to read it

cell = column+row; formula begins =; relative A1; absolute A1; function e.g. SUM(range)

How it works

- Identify whether the task is documents, slides, calculations, or structured records.
- Choose the matching component and its main data object.
- Describe the output that helps the user complete the task.

Diagram / process map / worked structure

```
raw rows -> validated table -> formula/summary -> chart/report
```

QUICK RECALL

Keep this in mind: Distinguish value, formula, cell reference, and displayed result in a sample worksheet.

TOPIC 44 / 48 | MS-Office

EXAMPLE, COMMON ERRORS AND EXAM HELP

MS-Excel

Worked example / application

If marks are in B2:B6, =AVERAGE(B2:B6) calculates their mean. A chart can visualize changes by category or time.

Important distinction / practical note

A spreadsheet is not automatically a relational database; avoid mixing multiple unrelated facts in one cell or relying on unvalidated manual entries.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Distinguish value, formula, cell reference, and displayed result in a sample worksheet.
- Likely follow-up: A spreadsheet is not automatically a relational database; avoid mixing multiple unrelated facts in one cell or relying on unvalidated manual entries.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Distinguish value, formula, cell reference, and displayed result in a sample worksheet.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 45 / 48 | MS-Office

CONCEPT AND DEFINITIONS

MS-Access

Simple definition

Access is a desktop relational database application for organizing related records into tables and working with them through queries, forms, and reports.

Formal / technical definition

A database application such as Microsoft Access manages relational tables with fields/records, keys and relationships, queries, forms for data entry, reports for presentation, and optional automation. Capabilities depend on product version and deployment.

Notation and how to read it

Access objects: tables | relationships | queries | forms | reports | macros/VBA (version-dependent)

How it works

- Identify whether the task is documents, slides, calculations, or structured records.
- Choose the matching component and its main data object.
- Describe the output that helps the user complete the task.

Diagram / process map / worked structure

```
tables store -> relationships connect -> query retrieves -> form edits -> report presents
```

QUICK RECALL

Keep this in mind: Explain the role of tables, queries, forms, and reports in one example database.

TOPIC 45 / 48 | MS-Office

EXAMPLE, COMMON ERRORS AND EXAM HELP

MS-Access

Worked example / application

A student database can store Student and Course tables; an Enrollment table with student/course keys links them; a query lists each student's courses; a form supports entry.

Important distinction / practical note

A form is an interface, not a replacement for database constraints; enforce keys and relationships in the data design.

Common mistakes to avoid

- Tables store structured records; forms provide an interface and do not replace sound table design.
- Access product capabilities vary by version and configuration.

Quick viva cues

- Core viva answer: Explain the role of tables, queries, forms, and reports in one example database.
- Likely follow-up: A form is an interface, not a replacement for database constraints; enforce keys and relationships in the data design.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Explain the role of tables, queries, forms, and reports in one example database.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 46 / 48 | Computer Languages
CONCEPT AND DEFINITIONS

Computer Languages: Introduction and Classification

Simple definition

A computer language is a formal way to express instructions and data operations so a computer system can carry them out.

Formal / technical definition

Programming languages define syntax and semantics for expressing computations. A common classification is machine language, assembly language, and high-level languages; another axis distinguishes procedural, object-oriented, functional, declarative, and scripting paradigms.

Notation and how to read it

low-level: machine/assembly; high-level: human-oriented abstractions; paradigms describe programming style, not level alone

How it works

- State the classification axis (purpose, size/capability, or mode of use).
- Describe the defining features of each category.
- Classify a real example only after naming the chosen axis.

Diagram / process map / worked structure

```
problem statement -> language source -> translation/runtime -> machine execution
```

QUICK RECALL

Keep this in mind: Classify examples by abstraction level and separately by paradigm where appropriate.

TOPIC 46 / 48 | Computer Languages
EXAMPLE, COMMON ERRORS AND EXAM HELP

Computer Languages: Introduction and Classification

Worked example / application

Machine instruction: bit/opcode pattern. Assembly: ADD R1,R2. High-level: total = a + b. Each expresses related computation at a different abstraction level.

Important distinction / practical note

Classification schemes answer different questions; a high-level language may support several paradigms.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Classify examples by abstraction level and separately by paradigm where appropriate.
- Likely follow-up: Classification schemes answer different questions; a high-level language may support several paradigms.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Classify examples by abstraction level and separately by paradigm where appropriate.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 47 / 48 | Computer Languages CONCEPT AND DEFINITIONS

Advantages and Limitations of Language Levels

Simple definition

Low-level languages provide close hardware control; high-level languages make many programs easier to write and maintain.

Formal / technical definition

Machine language executes directly on a processor but is difficult to write, debug, and port. Assembly provides symbolic hardware-oriented instructions but is architecture-specific. High-level languages improve abstraction, readability, and portability, but require translation/runtime support and may constrain low-level control or performance tuning.

Notation and how to read it

trade-off dimensions: readability | control | portability | performance | development effort | hardware dependence

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Diagram / process map / worked structure

```
More abstraction -> often easier development/portability
Less abstraction -> finer control, greater machine dependence
```

QUICK RECALL

Keep this in mind: Compare machine, assembly, and high-level languages on at least three criteria.

TOPIC 47 / 48 | Computer Languages
EXAMPLE, COMMON ERRORS AND EXAM HELP

Advantages and Limitations of Language Levels

Worked example / application

An embedded device driver may need hardware-specific control; a business application may benefit from a high-level language's libraries and portability.

Important distinction / practical note

No level is universally best; suitability depends on performance, safety, target hardware, team, and maintenance needs.

Common mistakes to avoid

- Define the term before giving its example.
- Show the steps clearly and check units, labels, and the final result.

Quick viva cues

- Core viva answer: Compare machine, assembly, and high-level languages on at least three criteria.
- Likely follow-up: No level is universally best; suitability depends on performance, safety, target hardware, team, and maintenance needs.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Compare machine, assembly, and high-level languages on at least three criteria.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

TOPIC 48 / 48 | Computer Languages
CONCEPT AND DEFINITIONS

Language Translators

Simple definition

A translator converts a program from one language or representation into another form the computer can execute or use.

Formal / technical definition

An assembler translates assembly mnemonics to machine/object code; a compiler translates source into target code before execution; an interpreter executes source/translated operations at runtime. Many systems combine compilation, bytecode, virtual machines, and JIT compilation.

Notation and how to read it

assembler: assembly -> machine/object; compiler: source -> target; interpreter: execute operations; hybrid implementations combine approaches

How it works

- Identify the input language and target representation.
- Choose assembler, compiler, interpreter, or a hybrid path.
- Trace translation and execution separately; note when they occur.

Diagram / process map / worked structure

```
source -> translation/interpretation -> execution environment -> output
```

QUICK RECALL

Keep this in mind: Define assembler, compiler, and interpreter, then give one distinction about when translation/execution occurs.

TOPIC 48 / 48 | Computer Languages
EXAMPLE, COMMON ERRORS AND EXAM HELP

Language Translators

Worked example / application

C source -> compiler -> object/executable; assembly -> assembler -> machine code; a Python interpreter executes program operations; Java source -> bytecode compiler -> JVM execution/JIT.

Important distinction / practical note

The common textbook compiler/interpreter contrast is simplified; modern runtimes may compile and interpret in stages.

Common mistakes to avoid

- The compiler/interpreter distinction is simplified in many textbooks.
- Modern systems may use bytecode, virtual machines, interpretation, and just-in-time compilation together.

Quick viva cues

- Core viva answer: Define assembler, compiler, and interpreter, then give one distinction about when translation/execution occurs.
- Likely follow-up: The common textbook compiler/interpreter contrast is simplified; modern runtimes may compile and interpret in stages.

Exam answer structure

- Begin with a clear definition and identify the key terms.
- Explain the rule or process, then show the example step by step.
- End with the result and one important limitation: Define assembler, compiler, and interpreter, then give one distinction about when translation/execution occurs.

QUICK RECALL

Close the notes and explain this topic aloud in your own words.

Rapid revision: one-line definitions

Introduction to computers

- Data and Information: Data are raw facts; information is data organized and interpreted so it helps answer a question.
- Computer: Definition and Main Features: A computer is an electronic, programmable system that accepts data, follows instructions, produces results, and can store them.
- Applications of Computers: Computers are used wherever information must be created, analyzed, communicated, controlled, or stored.
- Generations of Computers and Their Features: Computer generations are a teaching framework that groups eras by the main hardware technology used to build computers.

Basic Computer Organization

- Basic Operations of a Computer: A computer accepts input, keeps data/instructions in memory, processes data, gives output, and stores results for later.
- Anatomy of a Computer System: A computer system combines hardware, software, data, and users/procedures; the main hardware parts communicate to perform work.
- Input/Output Units and Their Functions: Input units send data or commands into a computer; output units present the computer's results in a form people or other devices can use.
- Storage Units and Their Functions: Storage keeps data and instructions.
- Types of Storage and Their Functions: Primary storage is used directly during processing; secondary storage keeps larger amounts of data more permanently.

Hardware & Software

- Hardware: Definition and Types: Hardware is the physical equipment that can be touched: processor, memory chips, keyboard, display, storage, and network devices.
- Software: Definition and Types: Software is the set of programs and related instructions that tell hardware what work to perform.
- Relationship Between Hardware and Software: Hardware provides the working parts; software provides instructions.

Classification of Computers

- Classification of Computers by Mode of Use and Features: Computers can be grouped by the kind of data they handle, the purpose they serve, their size/performance, and how many users or tasks they support.

Rapid revision: one-line definitions

Number System

- Number Systems: Introduction and Notation: A number system is a set of symbols and rules for writing quantities.
- Positional and Non-Positional Number Systems: In a positional system, a symbol's place changes its value.
- Decimal Number System (Base 10): Decimal uses ten digits, 0 through 9.
- Binary Number System (Base 2): Binary uses only 0 and 1.
- Octal Number System (Base 8): Octal uses digits 0 through 7.
- Hexadecimal Number System (Base 16): Hexadecimal uses 0-9 and A-F for values zero through fifteen.
- Decimal Integer to Binary, Octal, or Hexadecimal: Repeatedly divide the decimal integer by the target base.
- Binary, Octal, and Hexadecimal to Decimal: Multiply each digit by its place value and add the results.
- Binary-Octal-Hexadecimal Conversion by Grouping: Because octal is 2^3 and hexadecimal is 2^4, convert binary by grouping bits rather than doing long division.
- Conversions with Fractional Parts: To convert a decimal fraction to another base, multiply the fraction repeatedly by the target base; the whole-number parts become digits from left to right.

Computer Codes

- Computer Codes: Introduction: A computer code assigns bit patterns to symbols or values so that data can be stored, processed, and exchanged consistently.
- BCD (Binary Coded Decimal): BCD encodes each decimal digit separately using a four-bit pattern.
- EBCDIC: EBCDIC is an 8-bit character encoding developed for IBM systems.
- ASCII: ASCII is a character code for basic English letters, digits, punctuation, and control characters.

Operating System

- Operating System: Definition and Need: An operating system is the core software that manages computer resources and provides services so applications and users can use hardware conveniently.
- Logical Architecture of an Operating System: A useful OS picture has users and applications at the top, OS services/kernel in the middle, and hardware at the bottom.
- Basic Functions of an Operating System: An OS runs programs, shares resources, stores files, communicates with devices, and protects the system from errors or unauthorized use.
- Multiprogramming, Multitasking, and Multithreading: These terms describe ways an operating system keeps work moving: several programs in memory, several tasks sharing time, or several execution paths inside one program.

Rapid revision: one-line definitions

Algorithms

- Algorithms: Introduction and Definition: An algorithm is a clear, finite set of steps for solving a problem or producing a result.
- Algorithm Criteria and Notation: A good algorithm says exactly what to do, works for the allowed inputs, finishes, and can be carried out step by step.
- Types of Algorithms: Algorithms can be grouped by the strategy they use, such as trying all options, dividing a problem, using a greedy choice, or repeating improvements.

Internet

- Internet: Introduction: The Internet is a worldwide network of connected networks that exchange data using agreed communication protocols.
- Evolution of the Internet: The Internet grew from research networking into a global infrastructure for communication, information, commerce, and digital services.
- Basic Services of the Internet: Internet services are the practical activities people use networks for: messaging, file transfer, web access, calls, remote access, and collaboration.
- World Wide Web (WWW): The Web is a system of linked pages and resources identified by web addresses and accessed through browsers using web protocols.
- Web Browsers: A browser is client software for requesting, receiving, interpreting, and displaying Web content.
- Uses of the Internet: People use the Internet to communicate, learn, work, transact, create, access services, and share information.

MS-Office

- Design Goals of MS-Office: Office productivity suites aim to help users create, organize, calculate, present, and share work using related applications.
- MS-Word: Word is a word-processing application for creating, editing, formatting, reviewing, and printing text-centered documents.
- MS-PowerPoint: PowerPoint is presentation software for organizing a message into slides that combine text, visuals, data, and speaker delivery.
- MS-Excel: Excel is spreadsheet software for storing values in cells, calculating with formulas, summarizing data, and making charts.
- MS-Access: Access is a desktop relational database application for organizing related records into tables and working with them through queries, forms, and reports.

Computer Languages

- Computer Languages: Introduction and Classification: A computer language is a formal way to express instructions and data operations so a computer system can carry them out.
- Advantages and Limitations of Language Levels: Low-level languages provide close hardware control; high-level languages make many programs easier to write and maintain.
- Language Translators: A translator converts a program from one language or representation into another form the computer can execute or use.

High-yield ideas, formulas, and memory cues

Computer model and parts

- Core cycle: input -> processing -> output; memory/storage retain data and programs.
- CPU includes the control unit (directs operations), ALU (arithmetic/logic), and registers (very fast working values).
- Main memory holds active programs/data; secondary storage keeps data without power.
- Input devices send data/commands; output devices present results; some devices do both.
- Hardware is physical; software is instructions/data. Both are needed for useful operation.

Units and classification

- 1 byte = 8 bits. Common binary units: 1 KiB = 1024 B; 1 MiB = 1024 KiB.
- RAM is volatile working memory; ROM/firmware retains essential startup instructions; storage is persistent.
- General-purpose computers serve varied tasks; special-purpose systems are designed for a narrower task.
- Microcomputers, workstations, servers, mainframes, and supercomputers differ by capability and use; category boundaries evolve.
- Generation summaries commonly follow vacuum tubes -> transistors -> integrated circuits -> microprocessors; later-generation labels vary.

Exam comparison prompts

- Data vs information: raw recorded facts vs interpreted, useful meaning.
- RAM vs secondary storage: working speed/volatility vs persistence/capacity.
- Input vs output: direction relative to the computer's processing system.
- Hardware vs software: physical components vs programs and instructions.
- When classifying a machine, state the basis first: purpose, capability/size, or mode of use.

High-yield ideas, formulas, and memory cues

Place-value formulas

- For base b, integer digits d_n...d_0 represent sum(d_i x b^i). Every digit satisfies 0 <= d < b.
- For a fraction, digits after the radix point use b^-1, b^-2, b^-3, ...
- Integer to base b: repeated division; read remainders upward.
- Fraction to base b: repeated multiplication; read integer parts downward in the order generated.
- Binary to octal: group in 3s. Binary to hexadecimal: group in 4s, outward from the radix point.

Solved conversion checks

- (45)10 = (101101)2: 45=32+8+4+1.
- (101101)2 = 32+0+8+4+0+1 = (45)10.
- (725)8 = 7x64+2x8+5 = (469)10.
- (2F)16 = 2x16+15 = (47)10.
- (11010110)2 -> 11|010|110 -> (326)8; -> 1101|0110 -> (D6)16.

Codes and operating system

- 8421 BCD encodes each decimal digit in one 4-bit group; 59 -> 0101 1001.
- ASCII is a character code; standard ASCII uses 7-bit values. EBCDIC is a distinct 8-bit code family.
- OS acts as a resource manager and provides services/interface between applications and hardware.
- Multiprogramming keeps multiple jobs available; multitasking shares CPU time responsively; multithreading splits a process into execution paths.
- An OS's logical architecture is often explained as applications/user interface -> system-call boundary -> kernel -> hardware.

High-yield ideas, formulas, and memory cues

Algorithm checklist

- Input/output are specified; every step is definite; the process is finite and effective.
- Trace table: step | variables | condition | output. Use a small test case first.
- Sequence performs steps in order; selection chooses a branch; iteration repeats steps.
- Correctness: the steps produce the required output for valid inputs; termination: they eventually stop.

Internet and Web

- Internet: interconnected networks using shared communication protocols.
- Web: linked information resources identified by URIs/URLs, commonly accessed with HTTP(S) and browsers.
- Web page request path: URL -> DNS lookup -> HTTP(S) request -> server response -> browser render.
- Email, file transfer, calls, and Web access are services/applications that use Internet connectivity.
- A browser requests and displays Web content; a search engine indexes/searches content through a browser.

Office and languages

- Word: text documents; PowerPoint: slide presentations; Excel: grid-based calculations/data; Access: relational records, queries, forms, and reports.
- Machine language is processor-specific binary instructions; assembly uses symbolic machine-level operations; high-level languages offer more abstraction.
- Assembler translates assembly; compiler translates source into target form; interpreter executes program operations at runtime.
- Modern language implementations may combine ahead-of-time compilation, bytecode, interpretation, and JIT compilation.
- Choose the application by the task and the language level by the required balance of control, portability, readability, and performance.

Frequently asked exam and viva themes

MODULE-I - foundations and organization

- Data vs information; computer characteristics and limitations.
- Generation technology and the input-process-output-storage diagram.
- Hardware/software, memory/storage, I/O, and classification of computers.

MODULE-II - number systems, codes, and OS

- Place-value conversions with complete working and verification.
- BCD vs pure binary; ASCII vs EBCDIC.
- OS functions, logical architecture, multiprogramming, multitasking, and multithreading.

MODULE-III - algorithms, Internet, Office, language

- Algorithm criteria, notation, and a trace table.
- Internet vs Web vs browser; services and evolution.
- Office component selection; language levels and translator paths.

How to earn method marks

- Write the formula or definition before the working.
- Show units, base, labels, and intermediate steps.
- Use a neat diagram or trace table when it clarifies the answer.
- Check the final result and mention assumptions.

Viva response pattern

- Give one clear definition.
- Name the main parts or rule.
- Use a small example.
- State one useful comparison or limitation.

Last-minute accuracy checks

- Use the correct base and read remainders in the right direction.
- Do not confuse Internet with WWW or browser with search engine.
- Choose the Office component by the task.
- Use terminology carefully when conventions vary by text or product version.

Module 1 exam practice: easy to challenging

Frequently asked long-answer questions

- Explain the data-processing cycle with a neat labelled block diagram.
- Describe the generations of computers and compare their main technologies and characteristics.
- Explain the anatomy of a computer and the roles of the CPU, memory, input, output, and storage.
- Compare primary memory and secondary storage, including volatility, speed, and uses.
- Define hardware and software, classify their types, and explain their relationship.
- Classify computers using purpose and capability/mode of use, with suitable examples.

Short-answer / viva

- How are data different from information?
- What does the ALU do?
- Why is RAM called volatile?
- What are the four basic computer operations?
- What technology is associated with the third generation?
- Can a computer work usefully without software?

Easy

- Convert 5 GB to MB using decimal units, stating your convention.
- Name two input and two output devices.
- Give one example of system software and one application software.
- State two features of a computer and one limitation.

Moderate

- Draw a computer block diagram and trace a keyboard input through processing to display output.
- Compare RAM, ROM, cache, and secondary storage by role and persistence.
- Make a generation table with technology, one feature, and one representative development.
- Classify a laptop, ATM controller, server, and weather-prediction supercomputer; state your basis.

Challenging

- A lab has slow program loading and frequent loss of unsaved work. Identify which storage/memory roles matter and recommend changes with reasons.
- Explain why 'computers are always accurate' is an incomplete statement using input and program errors.
- Compare general-purpose and special-purpose computers using design goal, flexibility, and examples.

Answer cues

- Data are raw facts; information is interpreted data with context.
- CPU = control unit + ALU + registers (simplified model).
- RAM is volatile; secondary storage is persistent.
- Hardware is physical; software is instructions and data.
- Classification depends on the named axis; examples can fit more than one category.

Module 2 exam practice: easy to challenging

Frequently asked long-answer questions

- Explain positional notation and derive the place-value expansion for a number in base b.
- Convert decimal integers and fractions to binary, octal, and hexadecimal, showing each step.
- Convert binary, octal, and hexadecimal values to decimal using positional weights.
- Explain BCD, ASCII, and EBCDIC and compare their encoding approach and use.
- Draw and explain the logical architecture and basic functions of an operating system.
- Differentiate multiprogramming, multitasking, and multithreading with examples.

Short-answer / viva

- What is the radix of a number system?
- Why can a base-2 number use only 0 and 1?
- What is the difference between BCD and pure binary?
- How many bits are in standard ASCII?
- What is the main role of an operating system?
- How does multitasking differ from multithreading?

Easy

- Convert (29)10 to binary by repeated division.
- Convert (11011)2 to decimal using place values.
- Write decimal 74 in 8421 BCD.
- Convert (3A)16 to binary by grouping each hexadecimal digit.

Moderate

- Convert (156)10 to octal and verify by expansion.
- Convert (101101.101)2 to decimal.
- Convert (11011101)2 to octal and hexadecimal by grouping.
- Compare multiprogramming and multitasking in terms of CPU use and user response.

Challenging

- Convert (45.625)10 to binary; show integer and fractional methods separately.
- Explain why decimal 0.1 may have a repeating binary representation and give an approximation to four binary fractional places.
- A user opens a browser, edits a document, and switches tasks. Explain the OS roles involved without confusing a process with a thread.

Answer cues

- Value = sum(digit x base^position).
- Repeated-division remainders are read bottom to top.
- Fractional multiplication integer parts are read in generation order.
- BCD for 59 = 0101 1001; ordinary binary for 59 = 00111011.
- Standard ASCII is 7-bit; EBCDIC is a distinct code family.

Module 3 exam practice: easy to challenging

Frequently asked long-answer questions

- Define an algorithm and explain its criteria using a simple problem.
- Write an algorithm and flowchart or pseudocode to find the largest of three numbers.
- Describe Internet evolution, services, the Web, and browser roles; distinguish the terms.
- Explain the uses and basic services of the Internet with examples.
- Compare MS-Word, MS-PowerPoint, MS-Excel, and MS-Access by task and main data object.
- Classify computer languages and compare machine, assembly, and high-level languages and translators.

Short-answer / viva

- What makes a step unambiguous?
- What is the difference between the Internet and WWW?
- Is a browser the same as a search engine?
- Which Office component is most suitable for calculations?
- What does an assembler translate?
- What is the main difference between a compiler and interpreter in the basic model?

Easy

- Write three algorithm steps to find the sum of two input numbers.
- Name three Internet services apart from the Web.
- Match report, presentation, marks analysis, and customer records to Office components.
- Give one example each of machine-level, assembly, and high-level language.

Moderate

- Write an algorithm to determine whether an integer is even or odd, including input and output.
- Trace how a browser loads a URL and explain where DNS and HTTP(S) fit.
- Design pseudocode to calculate the average of five marks and display pass/fail.
- Compare compiler, interpreter, and assembler by input and output.

Challenging

- Design an algorithm to find the largest value in a list, covering the empty-list assumption and a trace table.
- Explain how an Office suite supports one project from data entry through analysis to a final presentation.
- A language runs on a virtual machine and uses JIT compilation. Explain why the simple compiler/interpreter contrast is not absolute.

Answer cues

- Algorithm: finite, clear steps that transform input into output.
- The Internet is network infrastructure; the Web is one information service on it.
- Word documents; PowerPoint slides; Excel workbooks; Access databases.
- Browser requests/renders Web content; search engine finds indexed information.
- Translator examples: assembler, compiler, interpreter; real runtimes can be hybrid.

Source notes and terminology

Reference points used for historical context

- Computer History Museum, computer timeline and technology milestones (vacuum tubes, transistors, integrated circuits, microprocessors): archive.computerhistory.org/resources/access/text/2018/08/102740419-05-01-acc.pdf
- Internet Society, A Brief History of the Internet (ARPANET, TCP/IP transition, and Internet development): internetsociety.org/internet/history-internet/brief-history-internet/
- World Wide Web Consortium, History of the Web (Web invention and early technologies): w3.org/about/history/

How to read textbook conventions

- Generation labels, approximate dates, and some computer classifications differ among introductory texts; the notes state the commonly taught technology-based sequence and flag variation.
- The terms kB/MB and KiB/MiB distinguish decimal and binary multiples. If a question gives a convention, follow that convention.
- ASCII and EBCDIC are historical character-code families. Exact mappings can depend on the specified encoding or code page.
- Office features vary by product version; the component descriptions focus on their core purpose.

QUICK RECALL

Use your syllabus wording first; use these notes to understand and explain each idea.

Created by Anurag Yadav

Email: annuanuragyadav908@gmail.com